

Lab 4: Simple Python Database Connections

DevOps

Name : Eman Khalid

Reg no : 23-BCS-026

Course Code: CS-649

Section: A

GitHub username: EmanKhalid27

Submitted To: Sir Waqas Saleem

Department: Computer Science

1. Introduction

This lab demonstrates how to connect Python applications to MySQL and MongoDB databases running in Docker. It covers hardcoded credentials, environment variables (`os.getenv`), and reading from a credentials file using `configparser`.

2. Task Evidence

Task 1: Fork and Clone

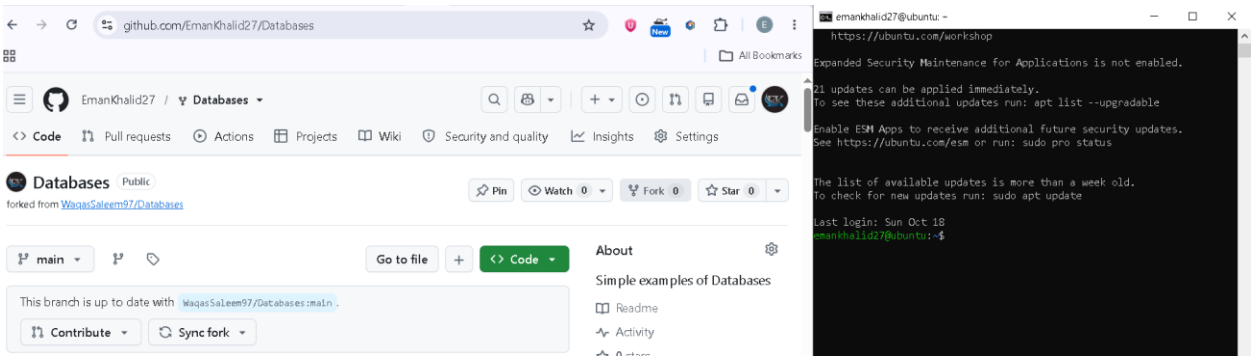


Figure 1.1: Forked Databases repository on GitHub. (`databases_fork.png`)

```
emankhalid27@ubuntu:~$ cd ~
t clone git@github.com:EmanKhalid27/Databases.git
cd Databases
lsemankhalid27@ubuntu:~$ git clone git@github.com:EmanKhalid27/Databases.git
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 15.00 KiB/s, done.
Resolving deltas: 100% (30/30), done.
emankhalid27@ubuntu:~$ cd Databases
emankhalid27@ubuntu:~/Databases$ ls
README.md      compose.yaml            db-init       requirements.txtVALIDATION.md  credentials.ini.example
  db_connect.py  tests
emankhalid27@ubuntu:~/Databases$
```

Figure 1.2: Successfully cloned the repository via SSH. (`databases_clone.png`)

Task 2: Install Docker and Python

```
emankhalid27@ubuntu:~$ git --version
git version 2.53.0
emankhalid27@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
emankhalid27@ubuntu:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
emankhalid27@ubuntu:~$
```

Figure 2.1: Verified Git, Docker, and Docker Compose versions. (`docker_installation.png`)

Task 3: Start Database Services

```
emankhalid27@ubuntu:~/Databases$ docker compose ps
NAME                    IMAGE      COMMAND                  SERVICE      CREATED          STATUS                    PORTS
databases-mongo-dev-1   mongo:7.0  "docker-entrypoint.s…"   mongo-dev    2 minutes ago    Up 2 minutes (healthy)    127.0.0.1:27017->27017/tcp
databases-mongo-prod-1  mongo:7.0  "docker-entrypoint.s…"   mongo-prod   30 minutes ago   Up 30 minutes (healthy)   127.0.0.1:27019->27017/tcp
databases-mongo-test-1  mongo:7.0  "docker-entrypoint.s…"   mongo-test   30 minutes ago   Up 30 minutes (healthy)   127.0.0.1:27018->27017/tcp
databases-mysql-1       mysql:8.4  "docker-entrypoint.s…"   mysql        30 minutes ago   Up 15 minutes (healthy)   33060/tcp, 127.0.0.1:3307->3306/tcp
emankhalid27@ubuntu:~/Databases$
```

Figure 3.1: All four database containers are running and healthy. (`database_services.png`)

```
emankhalid27@ubuntu:~/Databases$ docker compose port mysql 3306
17
docker compose port mongo-test 27017
docker compose port mongo-prod 27017127.0.0.1:3307
emankhalid27@ubuntu:~/Databases$ docker compose port mongo-dev 27017
127.0.0.1:27017
emankhalid27@ubuntu:~/Databases$ docker compose port mongo-test 27017
127.0.0.1:27018
emankhalid27@ubuntu:~/Databases$ docker compose port mongo-prod 27017
127.0.0.1:27019
emankhalid27@ubuntu:~/Databases$
```

Figure 3.2: Port mappings for MySQL and MongoDB environments. (`database_ports.png`)

Task 4: MySQL Hardcoded Connection

```
emankhalid27@ubuntu:~/Databases$ source .venv/bin/activate
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 4.1: Python script successfully connects to `school_mysql` on port 3307. (`mysql_hardcoded_connection.png`)

Task 5: MongoDB Hardcoded Connection

```
(.venv) emankhalid27@ubuntu:~/Databases$ nano db_connect.py
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 5.1: Python script successfully connects to `school_dev` on port 27017. (`mongo_hardcoded_connection.png`)

Task 6: MongoDB with `os.getenv()`

```
emankhalid27@ubuntu: ~/Databases
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 6.1: Connected to Dev environment using environment variables. (`mongo_env_dev.png`)

```
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 6.2: Connected to Test environment using environment variables. (`mongo_env_test.png`)

```
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 6.3: Connected to Prod environment using environment variables. (`mongo_env_prod.png`)

Task 7: MongoDB with Credentials File

```
(.venv) emankhalid27@ubuntu:~/Databases$ cp -n credentials.ini.example credentials.ini
cp: warning: behavior of -n is non-portable and may change in future; use --update=none instead
(.venv) emankhalid27@ubuntu:~/Databases$ ls credentials.ini
credentials.ini
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 7.1: Created `credentials.ini` file. (`credential_files_created.png`)

```
(.venv) emankhalid27@ubuntu:~/Databases$ chmod 600 credentials.ini
ini(.venv) emankhalid27@ubuntu:~/Databases$ stat -c '%a %n' credentials.ini
600 credentials.ini
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 7.2: Restricted file permissions to `600`. (`credential_file_permissions.png`)

```
emankhalid27@ubuntu: ~/Databases
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 7.3: Connected to Dev environment using the credentials file. (`mongo_file_dev.png`)

```
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 7.4: Connected to Test environment using the credentials file. (`mongo_file_test.png`)

```
(.venv) emankhalid27@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_prod
Server: ('127.0.0.1', 27019)
Environment: prod
(.venv) emankhalid27@ubuntu:~/Databases$
```

Figure 7.5: Connected to Prod environment using the credentials file. (`mongo_file_prod.png`)

4. Conclusion

I successfully edited one Python file through four connection stages (MySQL hardcoded, MongoDB hardcoded, MongoDB with `os.getenv`, and MongoDB with a credentials file). The final `db_connect.py` uses the credentials-file version, and all database connections were verified against their respective environments.